

Benefits of Object-Orientation: A Critical Reflection on the Solar System Planets Solution

Object-orientation (OO) organises software around classes that encapsulate state and behaviour, rather than around loosely coupled procedures and global data. Meyer frames this as information hiding and modular design that localise change (Meyer, 1997). Booch likewise emphasises abstraction and encapsulation as means of managing complexity in evolving systems (Booch, 1994). In my solar-system planets solution I applied these ideas at a modest scale: three collaborating classes (`Planet`, `SolarSystem`, `PlanetApp`) separate domain data, collection logic, and console interaction. This reflection evaluates where OO helped, where it added overhead for a small console application, and what the design still lacks.

The `Planet` class binds name, mass, distance, and moons with methods that interpret that state:

```
class Planet:
    def __init__(self, name, mass_kg, distance_au, moons=None):
        self.name = name
        self.mass_kg = float(mass_kg)
        self.distance_au = float(distance_au)
        self.moons = list(moons) if moons is not None else []

    def describe(self) -> str:
        # formats mass, distance, and moon list
        ...
```

Encapsulation here is practically useful: callers request a description or moon count without duplicating formatting rules across the menu. Copying the moons list on construction reduces accidental shared mutation--a small but deliberate defensive choice. Yet the trade-off is incomplete: attributes remain public, so clients can still assign nonsensical masses or replace the moons list after construction. True information hiding would use private fields or validated properties (Meyer, 1997). For this referral console program that extra ceremony would have been disproportionate; for a library shared across teams it would not. OO therefore helped, but only to the depth the problem justified.

`SolarSystem` loads JSON into `Planet` instances and offers case-insensitive lookup:

```
for entry in planets_raw:
    planet = Planet(
        name=entry["name"], mass_kg=entry["mass_kg"],
        distance_au=entry["distance_au"], moons=entry.get("moons", []),
    )
    self._planets[planet.name.casefold()] = planet

def get_planet(self, name: str) -> Planet:
    key = name.strip().casefold()
    if key not in self._planets:
        raise PlanetNotFoundError(f"Unknown planet: {name!r}")
    return self._planets[key]
```

This illustrates a central OO benefit: the collection owns construction and lookup policy. Menu code never parses JSON or manipulates dictionaries of dictionaries. Sommerville argues that modular decomposition improves maintainability when responsibilities are clear and change is localised (Sommerville, 2016). Altering the data source, validation of JSON shape, or lookup rules touches `SolarSystem` alone. The private `_planets` map is a stronger encapsulation example than `Planet`'s open fields, and the custom `PlanetNotFound`

Error makes failure modes explicit rather than returning sentinel values.

PlanetApp validates input and delegates queries, keeping presentation separate from domain logic:

```
class PlanetApp:
    def __init__(self, solar_system: SolarSystem):
        self.system = solar_system

    def _ask_planet_name(self) -> str | None:
        name = input("Enter planet name: ").strip()
        if name == "":
            print("Error: planet name cannot be empty.\n")
            return None
        return name
```

Separation of concerns let me unit-test Planet and SolarSystem without simulating stdin or printing menus. That is a concrete payoff of OO for quality assurance: classes with narrow interfaces are easier to exercise in isolation than a monolithic script, and regressions in lookup or formatting can be caught before the interactive loop is run. Python's class model supports this style directly through methods, composition, and exceptions (Python Software Foundation, 2024). Dependency injection of an already-loaded SolarSystem into PlanetApp further decouples I/O from data loading, which also made fixture-based tests straightforward.

Critical evaluation matters as much as praise. For eight planets and a text menu, a procedural script with a list of dictionaries would have been shorter and equally correct at runtime. OO's ceremony--constructors, methods, custom exceptions, three modules--pays dividends when behaviour grows (new queries, alternate interfaces, richer validation). At the current size, the main gains are pedagogical clarity, maintainability of responsibilities, and testability, not performance or cross-project reuse. There is also a cognitive cost: readers must navigate class boundaries and exception types before understanding a simple lookup. Design Patterns catalogue recurring collaborations at a larger grain (Gamma et al., 1995); my design uses only a simple facade-like application over a repository-like system. Inheritance is unused; composition suffices and is preferable here, avoiding fragile hierarchies that would invent subtypes without behavioural variation. Limitations remain: no polymorphic planet subtypes, no immutable value objects, weak attribute protection on Planet, and a hard-coded JSON loader rather than an abstract data source. I would strengthen encapsulation and inject a loader interface if the assessment required extensibility beyond a fixed file.

In summary, OO helped me model the domain, isolate change, and test without the UI. I do not claim it was strictly necessary for this referral exercise; I claim it was a proportionate preparation for larger systems where the same habits of abstraction, encapsulation, and modularity scale further than a single script ever could. The disciplined critique--recognising overhead as well as benefit--is itself part of postgraduate practice in software design.

References

- Booch, G. (1994). Object-oriented analysis and design with applications (2nd ed.). Benjamin/Cummings.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Meyer, B. (1997). Object-oriented software construction (2nd ed.). Prentice Hall.
- Python Software Foundation. (2024). The Python language reference. <https://docs.python.org/3/reference/>
- Sommerville, I. (2016). Software engineering (10th ed.). Pearson.